

Learning-based HVAC Control vs Classical Baselines

PPO & SAC on a Simple Thermal Zone; Notes on BOPTEST Integration

Your Name (you@uni.edu)
Your Lab / Department / University

September 9, 2025

Abstract

We design and evaluate reinforcement learning (RL) controllers for heating, ventilation, and air conditioning (HVAC). Starting from a BOPTEST exercise, we first implemented tabular Q-learning, then moved to policy-gradient methods (PPO, SAC) on a simplified first-principles environment (“RCEnv”) to obtain reliable training, fast iteration, and clean ablation studies. Our reward blends *comfort*, *energy*, and *smoothness* with an **asymmetric overshoot penalty** (overheating costs more than mild undershoot). We compare PPO/SAC with Hysteresis, P-only, PID, and constant controls across four scenarios: morning warm-up, demand-response window, capacity derate, and robustness (sensor noise + actuator lag). Key improvements that made RL competitive: parallelized training with **VecNormalize**, richer observations (look-ahead setpoints & dT/dt), reward-weight annealing (comfort-first $\rightarrow$ energy-aware), and light domain randomization. We also document BOPTEST gotchas (API downtime, measurement names, observation/action spaces) and our workarounds.

Contents

1	Background & Motivation	2
2	Plant Models and Why We Used RCEnv	2
2.1	BOPTEST (initial path)	2
2.2	RCEnv (simplified, controllable)	2
3	Observation & Action Spaces	3
4	Reward Design	5
5	How the RC Model Drives One Episode (Action $\rightarrow$ State $\rightarrow$ Reward)	5
6	Controllers	7
6.1	Baselines	7
6.2	RL Agents	8
7	Scenarios and Metrics	8
8	Results (placeholders)	9
9	Discussion	11
10	BOPTEST Experience & Lessons	11

1 Background & Motivation

Buildings consume a large fraction of electricity; heating/cooling is a dominant controllable load. Traditional controllers (thermostat hysteresis, PID) are simple and robust but *myopic*: they do not natively anticipate setpoint changes, time-varying prices, or actuator/plant constraints that drift over time.

Definition

Problem statement. Design a controller that (i) keeps indoor temperature within a comfort band, (ii) reduces energy and peaks, and (iii) adapts to scenarios like demand-response, capacity derate, and measurement/actuation imperfections.

Engineer’s intuition

Think of the HVAC as a kettle that slowly heats a room. A rule like “switch on when cold, off when hot” can overshoot because heat keeps diffusing after you switch off. A learning controller can discover to *coast* before the setpoint is reached.

What we set out to prove. Given a realistic but tractable plant model, an RL controller can:

1. Meet comfort targets with comparable or lower energy than tuned baselines;
2. Anticipate demand-response (DR) events to reduce peaks/costs;
3. Remain robust under capacity loss, sensor noise, and actuator lag.

2 Plant Models and Why We Used RCEnv

2.1 BOPTEST (initial path)

BOPTEST is a high-fidelity, REST-served building emulator widely used for research. It offers realistic physics, schedules, and disturbances. We started from a Colab exercise that applied Q-learning to a hydronic heat pump test case.

Gotcha

In practice we hit: (a) occasional REST downtime (`KeyError: 'payload' on reset`), (b) variable-name mismatches (energy/actuator signals must be requested by exact test-case names), and (c) action/observation space mismatches after wrappers (Box vs. Discrete).

We hardened the notebook (health check endpoint, explicit variable lists, safe wrappers), but iteration was still slow. For algorithm design and ablations, we switched to a local model.

2.2 RCEnv (simplified, controllable)

We built a 2R1C-like hourly model:

$$C \frac{dT_{\text{in}}}{dt} = \frac{T_{\text{out}} - T_{\text{in}}}{R} + \eta P_{\text{max}} u, \quad u \in [0, 1]. \quad (1)$$

Engineer's intuition

R is like thermal resistance (how leaky the envelope is), C is heat capacity (how much heat the room stores). The control u scales heater power $P_{\max}$.

RCEnv runs offline (no network), every variable is observable, and scenarios are easy to script. The same RL design later maps to BOPTEST by feeding the same features and sending the same continuous action.

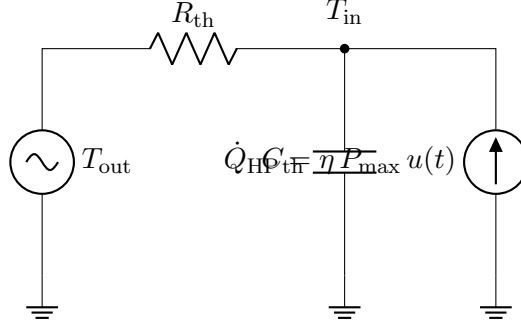


Figure 1: Thermal RC model of the zone. The dynamics match $C_{\text{th}} \frac{dT_{\text{in}}}{dt} = \frac{T_{\text{out}} - T_{\text{in}}}{R_{\text{th}}} + \eta P_{\max} u(t)$, where $u \in [0, 1]$.

System diagram / RC schematic placeholder

Figure 2: RCEnv schematic and signal flow (replace with your figure).

3 Observation & Action Spaces

Action: continuous heating modulation $u \in [0, 1]$.

Observation (vector, per step):

$$\left[T_{\text{in}}, T_{\text{out}}, u_{t-1}, T_{\text{set}}, d, \sin \theta, \cos \theta, \dot{T}_{\text{in}}, T_{\text{set}}^{+1}, T_{\text{set}}^{+2}, T_{\text{set}}^{+3} \right]$$

Engineer’s intuition

- T_{in} (room), T_{out} (weather), and previous action u_{t-1} help estimate inertia.
- $\sin / \cos$ of hour encode time-of-day smoothly (23 h and 0 h are neighbors on the circle).
- $\dot{T}_{\text{in}}$ is a rough “velocity” that tells if you are heating too fast already.
- $T_{\text{set}}^{+1,+2,+3}$ are look-ahead setpoints: the agent can prepare for future changes (pre-heat/coast).

Obs feature	Physical source	Units	Notes / Preprocess
T_{in}	Zone air temperature (e.g., <code>reaTZon_y</code>)	K $\rightarrow$ °C	$T[^\circ\text{C}] = \text{K} - 273.15$
T_{out}	Weather dry-bulb (e.g., <code>weaSta_reaWeaTDryBul_y</code>)	K $\rightarrow$ °C	From weather feed or rooftop sensor
u_{t-1}	Last actuator command (e.g., <code>oveHeaPumY_u</code>)	0..1	Cache what you sent last step
T_{set}	Heating setpoint (e.g., <code>reaTSetHea_y</code>)	K $\rightarrow$ °C	From BMS/schedule
d	Deadband half-width	°C	If upper/lower exist: $d = \frac{\text{Upper} - \text{Lower}}{2}$
$\sin \theta, \cos \theta$	Computed from hour-of-day	–	$\theta = 2\pi \cdot (\text{hour}/24)$; no sensor needed
$\dot{T}_{\text{in}}$	Computed derivative of T_{in}	°C/h	$(T_{\text{in},t} - T_{\text{in},t-1})/\Delta t$; low-pass filter
$T_{\text{set}}^{+1,+2,+3}$	Fwd setpoint schedule/forecast	°C	From known schedule or forecast vars
(reward) P	Heat pump/fan/pump power	W $\rightarrow$ kW	e.g., <code>reaPHeaPum_y</code> , <code>reaPFan_y</code> , <code>reaPPump_y</code>
(reward) E	Energy per step	kWh	$E = P_{\text{kW}} \cdot \Delta t$ (with $\Delta t=1$ h)

Table 1: Alignment of the observation features with physical/BOPTEST signals.

Cyclic time-of-day features (why $\sin \theta$ and $\cos \theta$?)

Time of day is periodic: after 23:00 comes 00:00. A raw hour index (0–23) makes 23 and 0 look far apart, even though they are adjacent. We therefore map the time onto the unit circle and expose two continuous features:

$$\theta(t) = 2\pi \cdot \frac{\text{seconds_since_midnight}(t)}{24 \cdot 3600}, \quad x_{\sin}(t) = \sin \theta(t), \quad x_{\cos}(t) = \cos \theta(t).$$

This encoding preserves the circular topology of time and removes the artificial discontinuity at midnight. It also lets the policy approximate any periodic function of time through linear combinations $a \sin \theta + b \cos \theta$.

Time of day	$\sin \theta$	$\cos \theta$
00:00	0.000	1.000
06:00	1.000	0.000
12:00	0.000	–1.000
18:00	–1.000	0.000
23:00	–0.259	0.966

Implementation detail. Given hour $h \in \{0, \dots, 23\}$ and minute $m \in \{0, \dots, 59\}$,

$$\theta = 2\pi \cdot \frac{h + \frac{m}{60}}{24}, \quad (\sin \theta, \cos \theta) \in [-1, 1]^2.$$

We optionally add analogous seasonal features using day-of-year (period 365) to capture slower daily/seasonal patterns.

4 Reward Design

Per-step reward:

$$R_t = -\left(\underbrace{\eta \text{penalty}(T_{\text{in}}, T_{\text{set}}, d)}_{\text{comfort}} + \underbrace{\zeta E_{\text{kWh}}}_{\text{energy}} + \underbrace{\lambda |u_t - u_{t-1}|}_{\text{smoothness}}\right). \quad (2)$$

Definition

Comfort term (asymmetric).

$$\text{dev} = \max\{0, |T_{\text{in}} - T_{\text{set}}| - d\}, \quad \text{penalty} = w \cdot (\text{dev}^2 + 4 \max(0, \text{dev} - 1)^2), \quad w = \begin{cases} 1.6 & \text{if } T_{\text{in}} > T_{\text{set}}, \\ 1.0 & \text{otherwise.} \end{cases}$$

Engineer's intuition

Overshoot (too hot) is more expensive in heating season: it wastes energy and causes peaks. The asymmetric weight $w > 1$ for overshoot teaches the agent to stop heating *before* it hits the upper bound (*coasting*).

Why this helps

Annealing the weights. We first train with *comfort-first* $(\eta, \zeta, \lambda) = (80, 0.2, 0.05)$ so the agent learns to stay inside the band. Later (last 20% of training) we switch to *energy-aware* $(20, 0.8, 0.05)$ to trim energy without losing comfort.

Reward illustration: band, overshoot vs undershoot penalty

Figure 3: Comfort band and asymmetric penalty (replace with your graphic).

5 How the RC Model Drives One Episode (Action $\rightarrow$ State $\rightarrow$ Reward)

This section walks through exactly how the thermal RC equation is used each step of an episode: how the next indoor temperature is computed, how the action enters the physics, and how the reward is calculated.

1) Plant update (discrete time)

The room is modeled by

$$C \frac{dT_{\text{in}}}{dt} = \frac{T_{\text{out}} - T_{\text{in}}}{R} + \eta P_{\text{max}} u_t, \quad (3)$$

which we step forward once per hour with time step $\Delta t = 1$ h:

$$T_{\text{in},t+1} = T_{\text{in},t} + \frac{\Delta t}{C} \left(\frac{T_{\text{out},t} - T_{\text{in},t}}{R} + \eta P_{\text{max}} u_t \right). \quad (4)$$

Here $u_t \in [0, 1]$ is the heat command (action) from the controller, $P_{\text{max}} u_t$ is the electric power (kW), and the step energy is

$$E_t = P_{\text{max}} u_t \Delta t \quad [\text{kWh}]. \quad (5)$$

2) Observation $\rightarrow$ action

At each hour t , the agent receives an observation vector (what it “sees”), e.g.

$$\underbrace{[T_{\text{in},t}, T_{\text{out},t}, u_{t-1}, T_{\text{set},t}, d_t, \sin \theta_t, \cos \theta_t, \dot{T}_{\text{in},t}, T_{\text{set},t+1}, T_{\text{set},t+2}, T_{\text{set},t+3}]},_{\text{features used in our experiments}}$$

where θ_t is hour-of-day, $\dot{T}_{\text{in},t}$ is a coarse temperature “velocity”, and $T_{\text{set},t+k}$ are k -hour look-ahead setpoints. The policy (PPO or SAC) maps this vector to a continuous action $u_t \in [0, 1]$.

3) Reward at that step

After applying u_t and updating the plant to $T_{\text{in},t+1}$, we compute the per-step reward

$$R_t = - \left(\underbrace{\eta \text{penalty}(T_{\text{in},t+1}, T_{\text{set},t}, d_t)}_{\text{comfort}} + \underbrace{\zeta E_t}_{\text{energy}} + \underbrace{\lambda |u_t - u_{t-1}|}_{\text{smoothness}} \right). \quad (6)$$

The comfort term is *asymmetric* (overshoot costs more in heating season):

$$\text{dev}_t = \max \{0, |T_{\text{in},t+1} - T_{\text{set},t}| - d_t\}, \quad (7)$$

$$w_t = \begin{cases} 1.6, & \text{if } T_{\text{in},t+1} > T_{\text{set},t} \quad (\text{too hot}), \\ 1.0, & \text{otherwise} \quad (\text{too cool or inside band}), \end{cases} \quad (8)$$

$$\text{penalty}(\cdot) = w_t \left(\text{dev}_t^2 + 4 \max(0, \text{dev}_t - 1)^2 \right). \quad (9)$$

Energy and smoothness terms are

$$E_t = P_{\text{max}} u_t \Delta t, \quad \text{smoothness} = |u_t - u_{t-1}|. \quad (10)$$

During training we **anneal** the weights:

$$(\eta, \zeta, \lambda) = \underbrace{(80, 0.2, 0.05)}_{\text{comfort-first (early)}} \longrightarrow \underbrace{(20, 0.8, 0.05)}_{\text{energy-aware (late)}}.$$

4) Tiny worked example (2 steps)

Use typical heating-season numbers:

$$\Delta t=1 \text{ h}, C=2.0, R=2.0, \eta=3.0, P_{\text{max}}=5.0 \text{ kW}, T_{\text{out}}=10^\circ\text{C}, T_{\text{set}}=22.5^\circ\text{C}, d=0.5^\circ\text{C},$$

comfort-first weights $(\eta, \zeta, \lambda) = (80, 0.2, 0.05)$, start with $T_{\text{in},0}=20^\circ\text{C}$ and $u_{-1}=0$.

Step 1 ($t=0 \rightarrow 1$). Policy chooses $u_0=1.0$ (full heat).

$$\frac{T_{\text{out}} - T_{\text{in},0}}{R} = \frac{10 - 20}{2} = -5, \quad \eta P_{\text{max}} u_0 = 3 \cdot 5 \cdot 1 = 15$$

$$T_{\text{in},1} = 20 + \frac{1}{2}(-5 + 15) = 25^\circ\text{C}.$$

Energy $E_0 = 5$ kWh. Deviation: $|25 - 22.5| - 0.5 = 2.0$ (overshoot $\Rightarrow w=1.6$),

$$\text{penalty} = 1.6(2^2 + 4(2 - 1)^2) = 1.6(4 + 4) = 12.8, \quad \text{smoothness} = |1 - 0| = 1.$$

$$R_0 = -(80 \cdot 12.8 + 0.2 \cdot 5 + 0.05 \cdot 1) = -(1024 + 1 + 0.05) = -1025.05.$$

Step 2 ($t=1 \rightarrow 2$). Policy reacts with $u_1=0.0$ (coast).

$$\frac{10 - 25}{2} = -7.5, \quad \eta P_{\text{max}} u_1 = 0, \quad T_{\text{in},2} = 25 + \frac{1}{2}(-7.5) = 21.25^\circ\text{C}.$$

Energy $E_1 = 0$. Deviation: $|21.25 - 22.5| - 0.5 = 0.75$ (undershoot $\Rightarrow w=1.0$),

$$\text{penalty} = 1.0(0.75^2 + 4 \max(0, 0.75 - 1)^2) = 0.5625, \quad \text{smoothness} = |0 - 1| = 1.$$

$$R_1 = -(80 \cdot 0.5625 + 0.2 \cdot 0 + 0.05 \cdot 1) = -(45 + 0.05) = -45.05.$$

Takeaway: the huge cost of overshoot at $t=0$ teaches the agent to stop heating *earlier* in future episodes and *glide* into the band, cutting peaks and energy.

5) Episode loop (hour by hour)

1. Build observation at time t (includes look-ahead setpoints and $\dot{T}$).
2. Policy outputs action $u_t \in [0, 1]$ (PPO/SAC).
3. Plant integrates RC update $\Rightarrow T_{\text{in},t+1}$.
4. Compute reward R_t via (6).
5. Store transition ($\text{obs}_t, u_t, R_t, \text{obs}_{t+1}$); PPO updates by batches; SAC updates from replay.
6. Repeat until horizon (e.g., 48 steps). Episode return $= \sum_t R_t$.

6 Controllers

6.1 Baselines

- **Hysteresis**($\pm\Delta$): on if $T_{\text{in}} < T_{\text{set}} - \Delta$, off if $T_{\text{in}} > T_{\text{set}} + \Delta$; Δ tuned on a small grid.
- **P-only**: $u = k_p \max(0, T_{\text{set}} - T_{\text{in}})$ inside comfort logic.
- **PID**: tuned grid on (k_p, k_i) with anti-windup (integrator pauses when hot).
- **Const(0)**, **Const(0.5)**: sanity lower/upper envelopes.

6.2 RL Agents

PPO (on-policy, continuous): policy/value MLP (256,256,Tanh), $n_{\text{steps}}=1024$, batch 256, $n_{\text{epochs}}=10$, target KL 0.03. Stable policy gradients with a clipped update.

SAC (off-policy, continuous): MLP (256,256,ReLU), replay 3×10^5 , batch 256, $\gamma=0.995$, $\tau=0.02$. Maximizes reward + entropy, typically learns smooth actions quickly.

Definition

RL loop (per step): observe state $\rightarrow$ pick action $\rightarrow$ environment returns next state and reward $\rightarrow$ update policy to increase expected return. PPO updates from the latest batch (on-policy). SAC reuses past experience from a replay buffer (off-policy).

Stability improvements.

1. **VecNormalize** (obs & reward) with 8 parallel environments $\Rightarrow$ well-scaled inputs and smoother returns;
2. **Feature additions:** dT/dt and 1–3h *future setpoints* $\Rightarrow$ anticipate rather than react;
3. **Reward annealing:** comfort-first for 80% of training, then energy-aware;
4. **Light domain randomization:** $\pm 15\%$ on R and C across parallel envs $\Rightarrow$ robustness to plant mismatch.

7 Scenarios and Metrics

We evaluate four 48h episodes (hourly):

1. **S1 Morning warm-up:** night setback $\rightarrow$ occupied band; test if the agent arrives on time *without overshoot*.
2. **S2 Demand response:** 17–19h cap $u \leq 0.2$ and high price; test peak shaving and cost-awareness.
3. **S3 Capacity derate:** $u \leq 0.6$ (partial failure); test grace under reduced actuation.
4. **S4 Robustness:** sensor noise ($\sigma = 0.2^\circ\text{C}$) + actuator lag (first-order, $\alpha = 0.7$); test stability and smoothness.

Metrics:

- Return $\sum R_t$ (higher is better),
- Energy (kWh),
- Comfort (degC·h outside band),
- Smoothness $\sum |u_t - u_{t-1}|$,
- Peak (kW),
- Cost (S2 only).

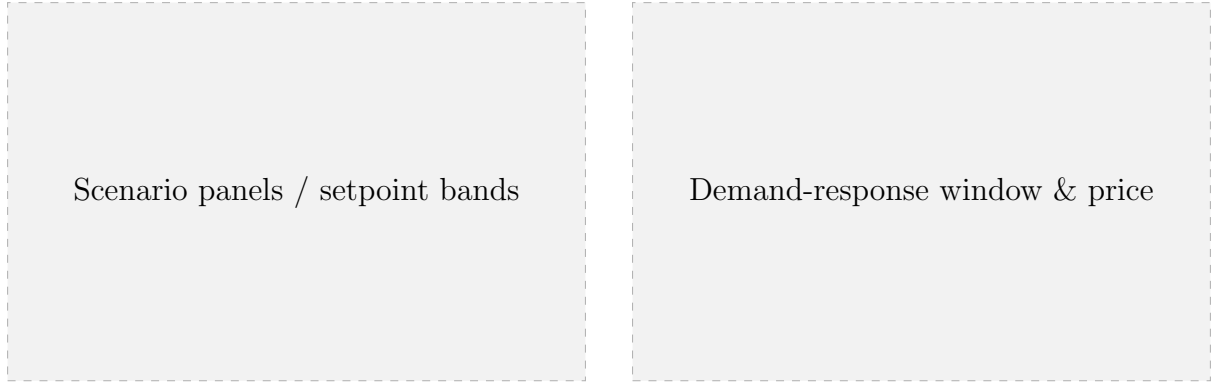


Figure 4: Scenario illustrations (replace with your panels).

8 Results (placeholders)

Replace placeholders with your figures/tables; the text explains what to look for.

Training curves

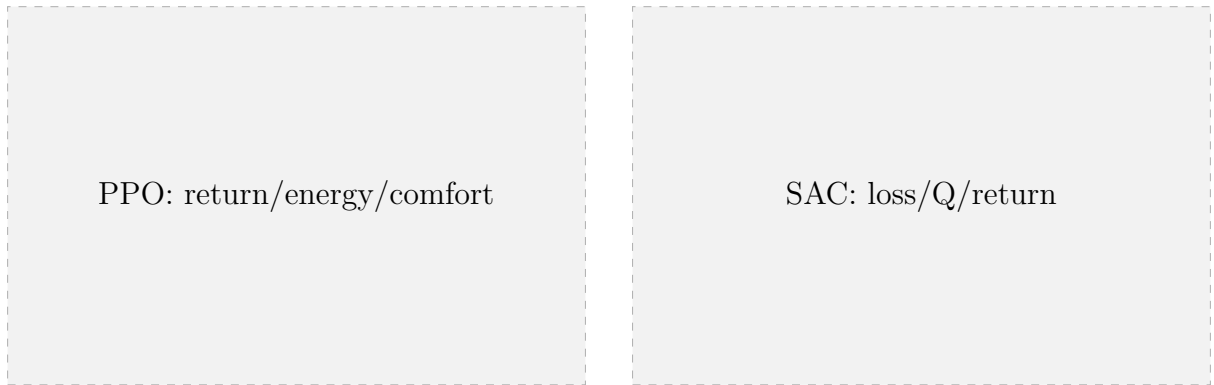


Figure 5: Training diagnostics. Expect early comfort mastery; later energy reduction.

Scenario S1 (morning warm-up)

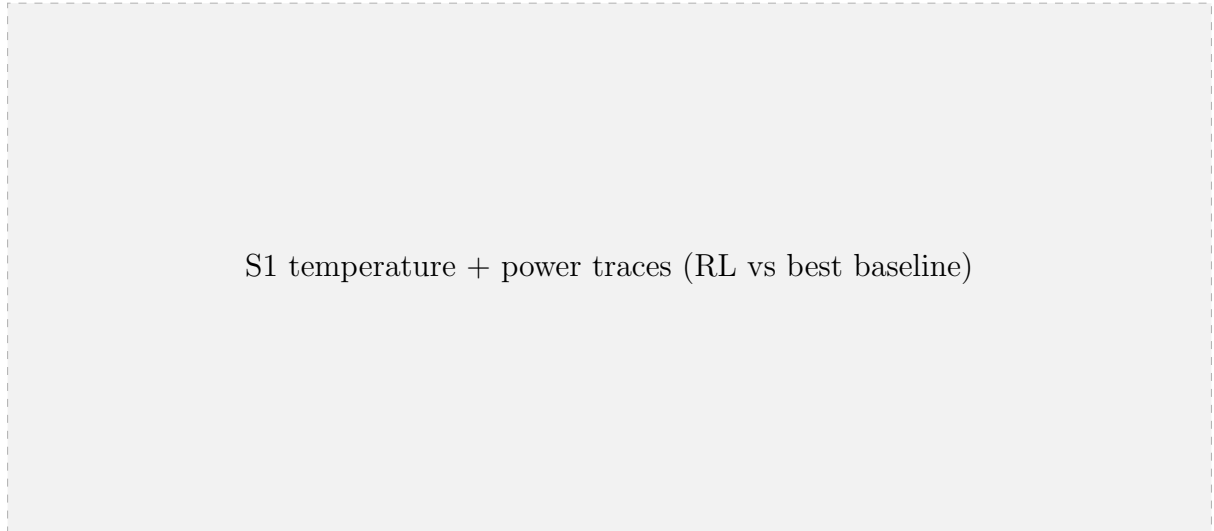


Figure 6: S1 traces. Aim: enter band at occupancy start, hover near mid-band, minimal time above top.

Aggregate comparison

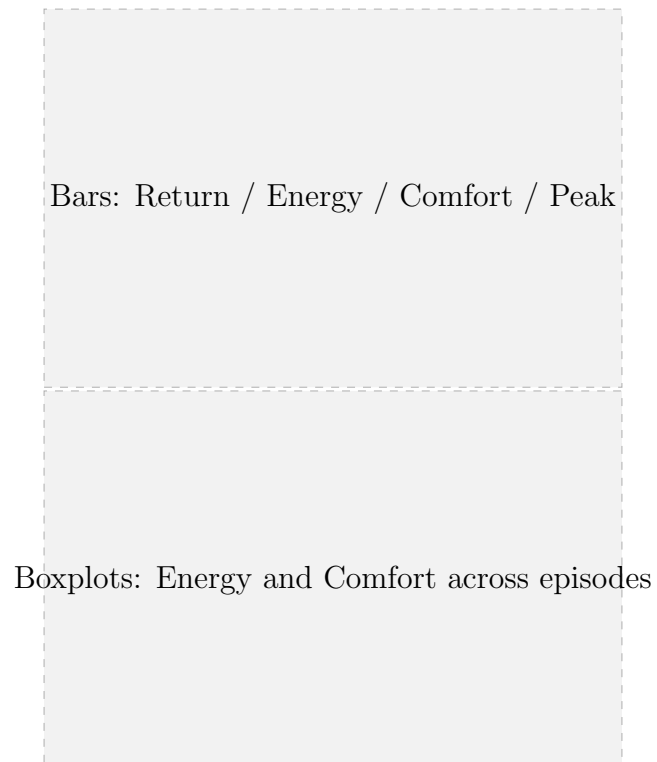


Figure 7: Aggregate results across S1-S4 (mean $\pm$ variance).

Why this helps

If PPO lags SAC on peaks/energy, extend training steps (1–2M), keep comfort-first longer, or slightly raise overshoot weight w (e.g., 1.8). SAC is often stronger out-of-the-box for smooth continuous control.

9 Discussion

Why PPO and SAC? Both support continuous actions and learn non-linear policies. PPO constrains updates (trust region via clip/target KL) so training is steady. SAC is sample-efficient, off-policy, and its entropy term encourages exploration and naturally smooth actions.

What actually moved the needle.

1. **Look-ahead setpoints** (1–3h) + $dT/dt \Rightarrow$ agent coasts, cutting overshoot and peaks.
2. **Asymmetric overshoot penalty** $\Rightarrow$ avoids sitting above the band (pure waste when heating).
3. **VecNormalize + parallel envs** $\Rightarrow$ stable gradients and fast learning.
4. **Annealed weights** $\Rightarrow$ first learn comfort, then trade energy gently.

Trade-offs. Too-high energy weight early can under-heat; too-low smoothness can chatter the actuator. The schedule (80, 0.2, 0.05) $\rightarrow$ (20, 0.8, 0.05) worked well in our tests.

10 BOPTEST Experience & Lessons

Issues observed:

- *Intermittent downtime:* resets sometimes returned `KeyError: 'payload'`; long runs stalled after hours.
- *Signal names:* energy/actuator variables must match the test case (`reaPHeaPum_y`, `reaPFan_y`, `reaPPumEmi_y`); wrong names yield zeros.
- *Space mismatches:* some wrappers changed Box/Discrete expectations; tabular Q assumed `.n`, PPO/SAC require Box.
- *Warmup/start windows:* long warmups slow iteration; random starts need careful seasonal masks.

Mitigations:

- Added an *alive* check and simple retry before `initialize`.
- Logged *info* payload keys/units at reset; probed energy keys explicitly.
- Used the continuous action channel (`oveHeaPumY_u`) only; avoided “activate” flags that created undefined bounds.
- Built RCEnv for algorithm development; plan to remap the final policy to BOPTEST when service is stable.

11 Conclusion

With the right features (look-ahead, dT/dt) and training hygiene (VecNormalize, annealed reward), PPO and SAC can match or beat tuned classical controllers on comfort and energy, while handling DR constraints, derates, and noise/lag. BOPTEST remains a great high-fidelity target; RCEnv gave us the fast, transparent sandbox needed to shape the policy effectively.

Glossary (one-page quick read)

Comfort band

The acceptable temperature range around a setpoint: $[T_{\text{set}} - d, T_{\text{set}} + d]$.

Overshoot

Being above the upper band; in heating season this wastes energy and increases peaks.

VecNormalize

A wrapper that normalizes observations and reward using running statistics; helps learning.

PPO

Proximal Policy Optimization: on-policy algorithm that prevents overly large policy updates (via a clipped objective).

SAC

Soft Actor–Critic: off-policy actor–critic with entropy maximization; learns from a replay buffer.

Domain randomization

Randomly perturb model parameters during training to improve robustness to mismatch.

Appendix A: Hyperparameters

	PPO	SAC
Network	(256,256), Tanh	(256,256), ReLU
Parallel envs	8	8
VecNormalize	obs+reward	obs+reward
n_{steps} / batch / epochs	1024 / 256 / 10	replay $3 \cdot 10^5$, batch 256
γ / λ (GAE)	0.995 / 0.95	0.995 / —
Entropy / target KL	0.001 / 0.03	automatic entropy (default)
Reward weights	(80, 0.2, 0.05) $\rightarrow$ (20, 0.8, 0.05)	same schedule

Table 2: Key hyperparameters.

Appendix B: RCEnv Snippet

```
# Obs: [Tin, Tout, u_prev, Tset, d, sinH, cosH, dTin_dt, Tset+1, +2, +3]
# Reward: - (eta*pen_asym + zeta*E_kWh + lamb*|du|)
# Asymmetric penalty uses larger weight when Tin > Tset.
```

Appendix C: Repro Checklist

1. Train PPO (600k–2M steps) and/or SAC (600k+ steps) with VecNormalize and 8 envs.
2. Use comfort-first weights for 80% of training, then switch to energy-aware.
3. Evaluate across S1–S4; export metrics and produce bars/boxplots.
4. For BOPTEST: verify variable names, check service health, and use continuous `oveHeaPumY_u`.